

Übungsblatt ÜE-05

Ausgabe: 19.11.2025
Abgabe: 29.11.2025
12:00 Uhr

Inventarisierungssystem

Hinweis:

- Die Abgabe bringt keine Bonuspunkte. Ihre Tutorin/Tutor erstellt Ihnen ein kurzes Feedback zur Abgabe. Das ganze dient also als Übung und Lernerfolgskontrolle für Sie!
- Sie dürfen für dieses Blatt alles nutzen, jedoch ist es nicht zielführend automatisch generierte Lösungen (z.B. über ein LLM) einfach zu kopieren und hochzuladen. **Bitte aber die Quellen angeben, also auch ob ein LLM bei der Erstellung unterstützt hat!**
- Geben Sie am Anfang aller Ihrer Quellcode Dateien (.py) die Variable `__author__` an (am Anfang des Quellcodes) – ist schon mal eine Übung, da dies für die Codereview-Aufgaben, welche für die Studienleistung relevant sind wichtig sein wird:
`__author__ = "<Matr-Nr>, <Nachname>"` also z.B.:
`__author__ = "1234567, Tolle"`
- Digitale Abgaben, die nicht im Format **.pdf** oder **.txt** für Texte oder **.py** für Code erfolgen, werden nicht überprüft. Bei Abgaben mehrerer Dateien müssen diese als **.zip** zusammengefasst werden.
- Immer wenn wir von Python Programm sprechen, meinen wir Python Code, der mindestens mit der Python Version 3.10 lauffähig ist. Bitte aktuell komplett ohne Imports arbeiten.
- Sollten Aufgabenstellungen nicht eindeutig sein, bitte durch Ihre eigenen Annahmen ergänzen und diese dokumentieren (z.B. in einem Kommentar).
- Achten Sie auch auf Vorgaben Ihrer Tutorin/Tutors. Hierfür auch in das Gruppenforum im Moodle-Kurs von EPR schauen!

Aktualisierung der Hinweise:

- Beim Programmieren und Kommentieren halten Sie sich bitte an die Regeln im Programmierhandbuch (Style Guide), siehe Moodle-Kurs. Im Zweifelsfall gilt PEP 8. ... möglichst selbst mit `pycodestyle` mal prüfen.

Aufgabe ÜE-05-1: Inventarisierungssystem

Entwickeln Sie ein Python-Programm, das dabei hilft, Gegenstände zu organisieren und zu verwalten, die an verschiedenen Orten wie einem Keller, einem Schrank oder anderen Lagerplätzen aufbewahrt werden. Diese Lagerorte sowie die darin enthaltenen Gegenstände sollen mit **Dictionaries** dargestellt werden. Gegenstände, die in Kisten oder Behältern aufbewahrt werden, sollen ebenfalls durch **verschachtelte Dictionaries** abgebildet werden, um die hierarchische Struktur der Lagerung zu berücksichtigen.

Das Programm soll die folgenden Funktionalitäten umfassen:

1. Hinzufügen von Gegenständen

Erstellen Sie eine Funktion, um neue Gegenstände an einen bestimmten Lagerort hinzuzufügen. Falls der Lagerort bereits existiert, sollen die Inhalte entsprechend aktualisiert werden. Falls er nicht existiert, soll er neu erstellt werden.

Das Bild soll nur illustrieren, was die Funktion machen sollte. Es handelt sich nicht um ein Programm. Auch sind die gezeigten Eigenschaften wie „Zustand“, „Model“ nicht vorgegeben. Entscheiden Sie selbst und erläutern Sie dies in Ihrer Dokumentation entsprechend.

```
Ursprünglich:
{}
Nach dem Aufruf der Funktion:
{
  "keller": {
    "Hammer": {
      "Anzahl": 5,
      "Zustand": "neu"
    },
    "Bett": {
      "Anzahl": 2,
      "Zustand": "alt"
    }
  },
  "garage": {
    "Auto": {
      "Anzahl": 1,
      "Zustand": "neu",
      "Model": "Porsche"
    },
    "garageitem": {
      "something": "value1"
    }
  }
}
```

2. Auflistung aller Gegenstände

Entwickeln Sie eine Funktion, die alle Gegenstände in allen Lagerorten alphabetisch auflistet.

Hier Lagerort: Keller und Garage

```
['Auto', 'Bett', 'Hammer', 'garageitem']
```

3. Suche nach Gegenständen

Schreiben Sie eine Funktion, mit der Sie nach einem bestimmten Gegenstand suchen können. Die Funktion soll angeben, in welchem Lagerort (oder auch Lagerorten, man kann ja auch gleiche Dinge an verschiedenen Orten lagern) oder Behälter sich der Gegenstand befindet.

Beispiel: Suche nach Bett: `(True, {'keller': {'Anzahl': 2, 'Zustand': 'alt'}})`

4. Zusammenfassung nach Lagerort

Fügen Sie eine Funktion hinzu, die eine Übersicht über die in einem oder mehreren angegebenen Lagerorten enthaltenen Gegenstände erstellt. Falls ein Lagerort nicht existiert, soll dies entsprechend angezeigt werden.

Beispiel: Zusammenfassung von: keller, garage, bedroom

```
{'keller': ['Hammer', 'Bett'], 'garage': ['Auto', 'garageitem'], 'bedroom': 'Location not found in inventory'}
```

5. Aktualisierung von Gegenständen

Entwickeln Sie eine Funktion, die es ermöglicht, die Eigenschaften eines Gegenstands zu aktualisieren. Demonstrieren Sie in diesem Zusammenhang, wie Änderungen in einem verschachtelten Dictionary (z. B. durch flache Kopien) auf das ursprüngliche Dictionary wirken können.

6. Erstellung/Anzeigen eines Backups

Schreiben Sie eine Funktion, die ein vollständiges Backup des Inventars erstellt (hier sollte man Deep Copy benutzen). Dieses Backup soll unabhängig vom ursprünglichen Inventar sein und zu einem späteren Zeitpunkt genutzt werden können (einfach gespeichert in einer anderen Variable, das Backup muss nicht in das Filesystem ausgelagert werden). Erfassen Sie dabei den Zeitpunkt der Backuperstellung (hier kann man **datetime.now()** nutzen). Über eine weitere Funktion soll der aktuelle Zustand zusammen mit dem Backup angezeigt werden können (s.u.).

Beispiel: Nach Erstellung des Backups wird ein Gegenstand aktualisiert. Im Bild werden die Ergebnisse angezeigt, wenn man sie ausgibt.

```
{
  "keller": {
    "Hammer": {
      "Anzahl": 5,
      "Zustand": "neu"
    },
    "Bett": {
      "Anzahl": 2,
      "Zustand": "alt"
    }
  },
  "garage": {
    "Auto": {
      "Anzahl": 1,
      "Zustand": "neu",
      "Model": "Mercedes",
      "Farbe": "Gelb"
    },
    "garageitem": {
      "something": "value1"
    }
  }
}

Backup: 2024-11-20 13:39:41
{
  "keller": {
    "Hammer": {
      "Anzahl": 5,
      "Zustand": "neu"
    },
    "Bett": {
      "Anzahl": 2,
      "Zustand": "alt"
    }
  },
  "garage": {
    "Auto": {
      "Anzahl": 1,
      "Zustand": "neu",
      "Model": "Porsche"
    },
    "garageitem": {
      "something": "value1"
    }
  }
}
```

Zusätzliche Hinweise:

- Lösungen sind eigenständig zu erstellen, ohne Zuhilfenahme von generativer KI (z. B. ChatGPT). In der Klausur wird dieses Hilfsmittel ebenfalls nicht zur Verfügung stehen.
- Eine Tutorin/Tutor oder die Dozentin/der Dozent kann ein Code-Review verlangen, bei dem Sie Ihren Code erklären und rechtfertigen müssen.
- Rekursive Ansätze zur Lösung der Aufgabe sind nicht erlaubt!
- **Docstrings und Testfälle nicht vergessen!**